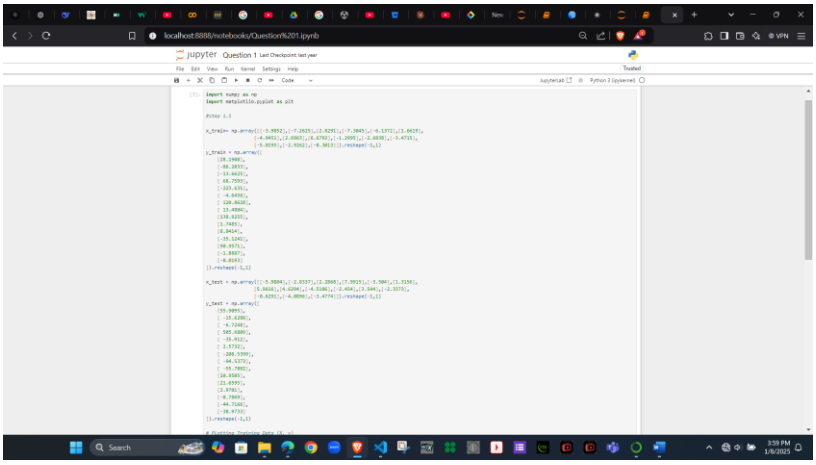
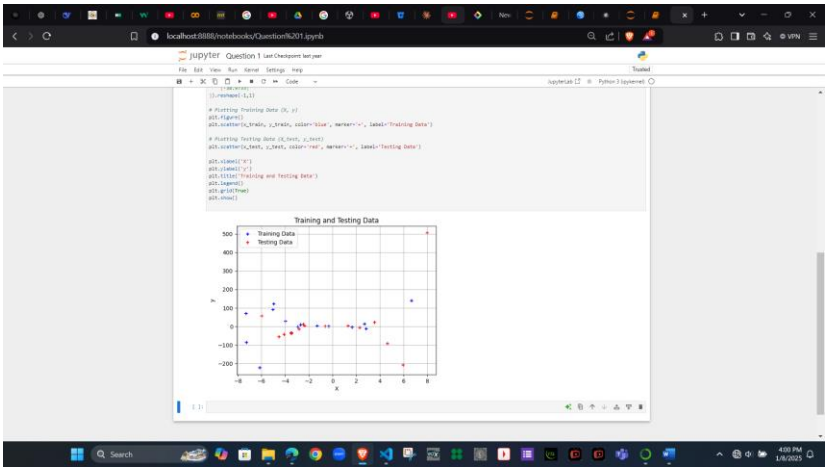
Question 1:

Step 1.1: Plotting Training and Testing Data





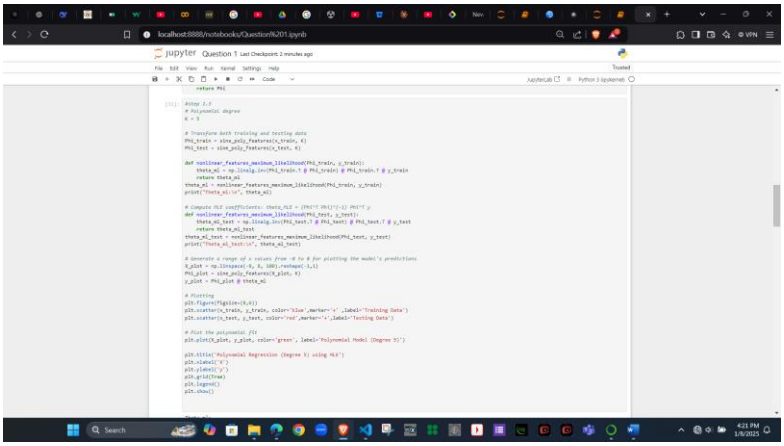
Step 1.2: Feature Transformation

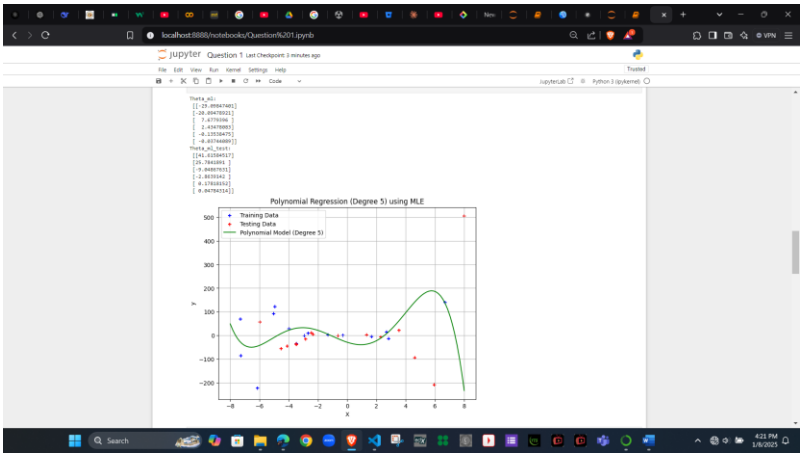
```python
#step 1.2
def sine_poly_features(X, K):
    # return np.hstack([X**i for i in range(K + 1)])
    X= X.flatten()
    N= X.shape[0]
    Phi= np.zeros((N,K+1))
    for k in range (K+1):
        Phi[:,k]= X**k
    return Phi
```

The purpose and the effect of this transformation:

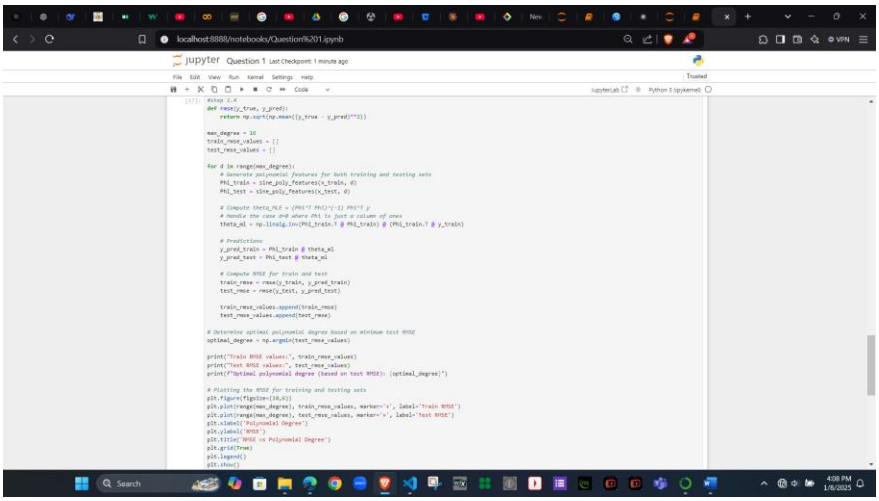
Polynomial transformation enables linear models to fit nonlinear data by adding higher-degree features. It improves flexibility but risks underfitting with low degrees or overfitting with high degrees. The right degree balances accuracy and complexity for better generalization.

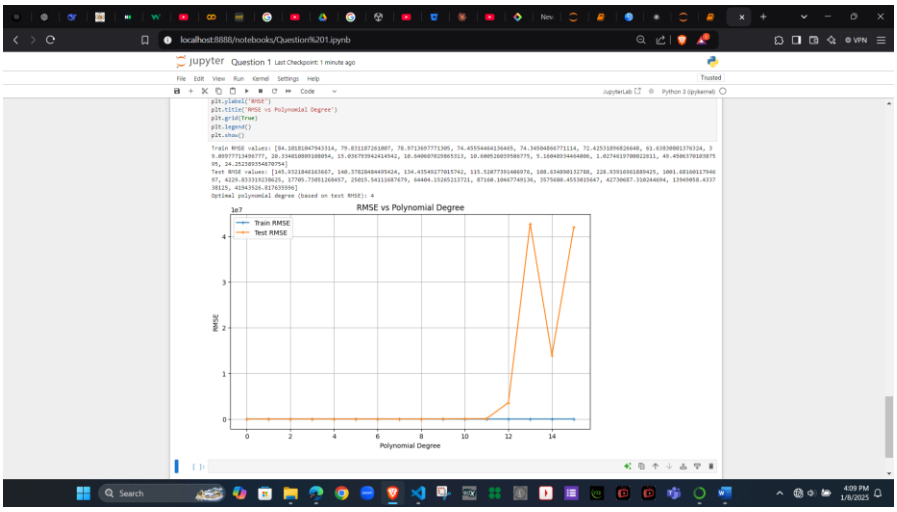
Step 1.3: Fitting the model using Maximum Likelihood





Step 1.4: Model Evaluation



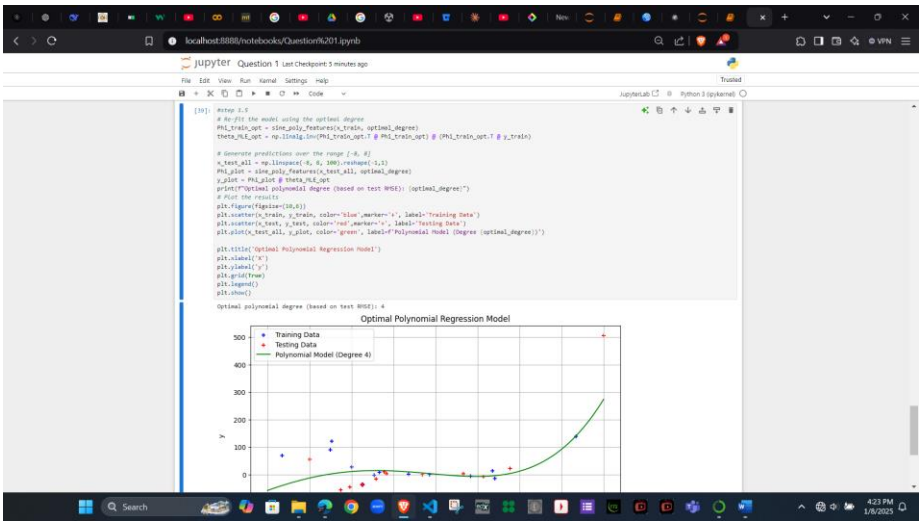


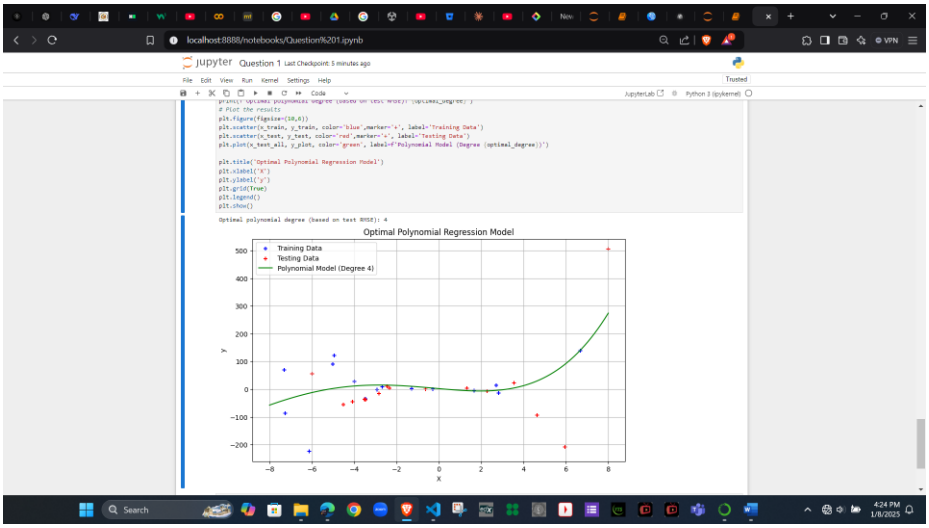
Train RMSE values: [84.10181047943314, 79.831187261007, 78.9713697771305, 74.45554464136465, 74.34504866771114, 72.42531896826648, 61.63830801376324, 39.09977713496777, 20.334010809108054, 15.036793942414542, 10.640607029865313, 10.600526039506775, 5.16048934464006, 1.0274619700022611, 49.450637010387595, 24.252389354870754]

Test RMSE values: [145.9321846163667, 140.37828484495424, 134.43549277015742, 115.52077391406976, 108.634890132788, 228.93916961889425, 1001.6816011794697, 4229.833319238625, 17705.73051268457, 25015.54111687679, 64404.15265213721, 87160.10467749136, 3575680.4553015647, 42730687.310244694, 13949058.433738125, 41943526.817635596]

Optimal polynomial degree (based on test RMSE): 4

Step 1.5: Selecting the best model





Why was this degree optimal?

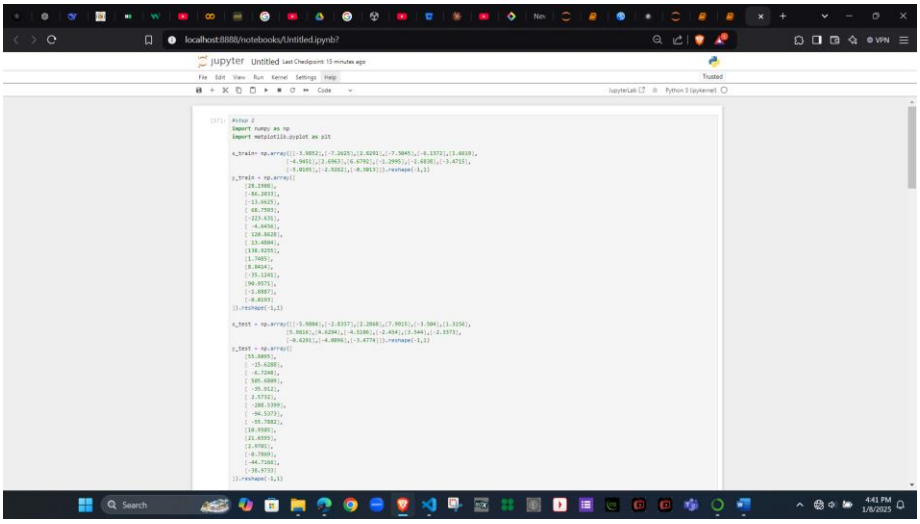
- The optimal degree minimizes the RMSE for the test dataset, striking a balance between underfitting (too simple) and overfitting (too complex).
- Degrees that are too low (e.g., 0 or 1) underfit, being too simple and cannot catch the small details in the data, so they don't fit well.
- Degrees that are too high (e.g., >10) overfit, capturing noise rather than the true relationship.

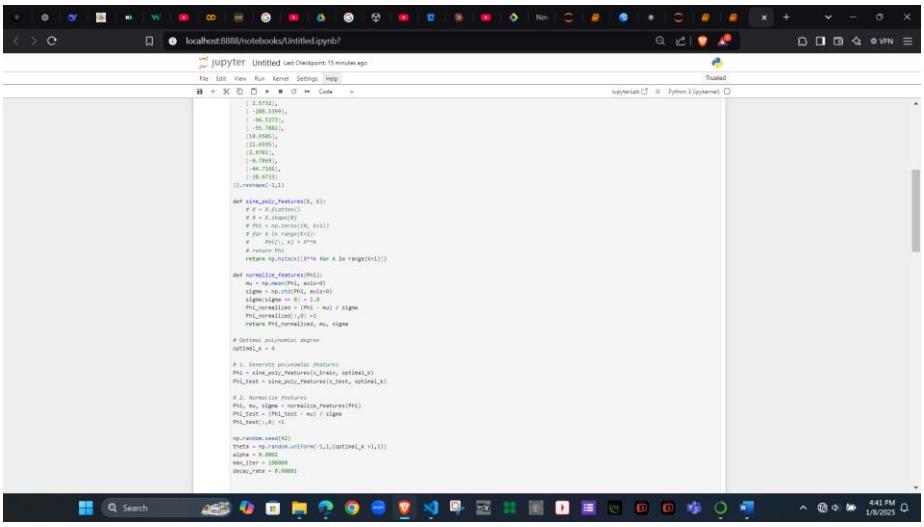
Model Complexity and Overfitting/Underfitting:

- Lower degrees: Underfitting, as the model cannot capture complex patterns.
- Higher degrees: Overfitting, as the model tries to fit every data point, including noise.
- Optimal degree: Provides the best generalization for unseen data (test set).

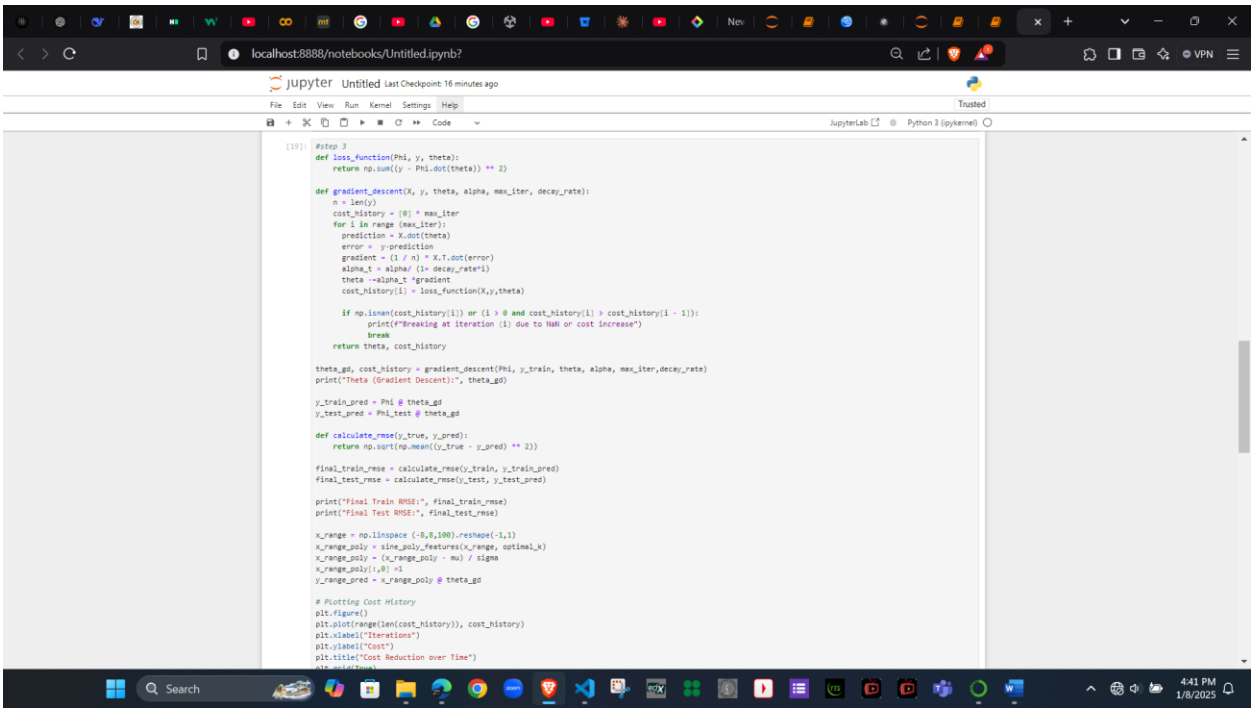
Question 2:

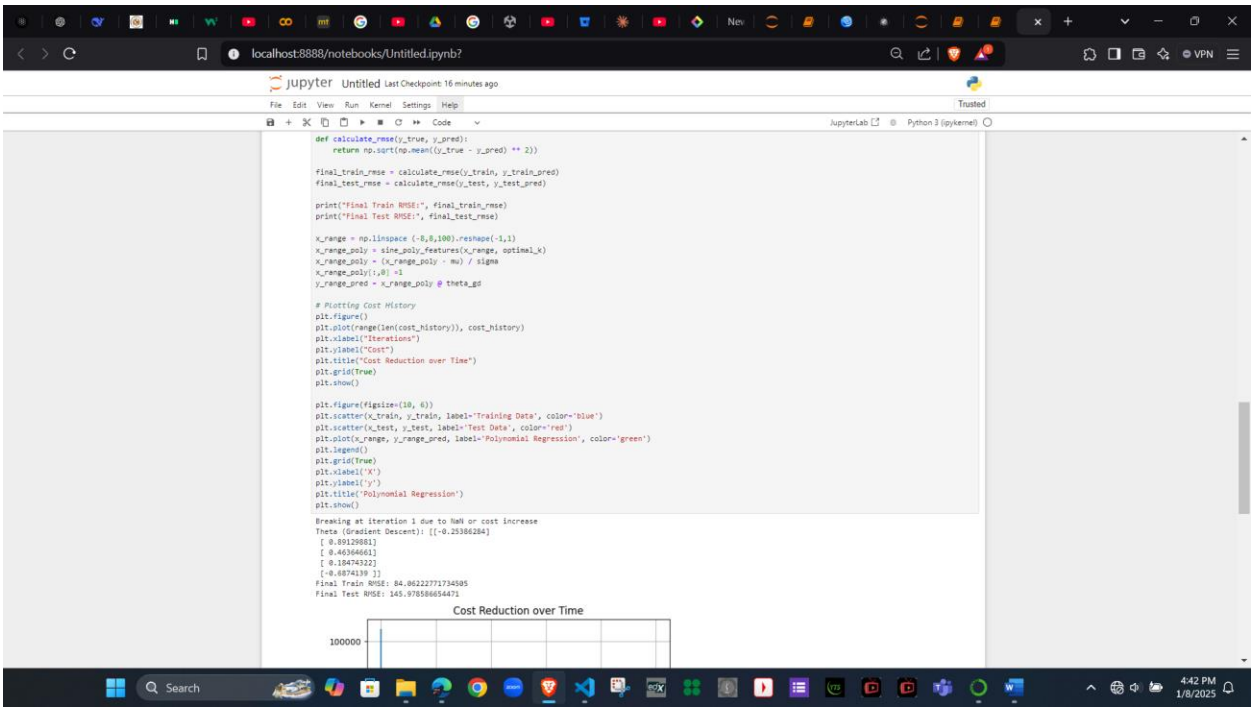
Step 2: Feature transformation

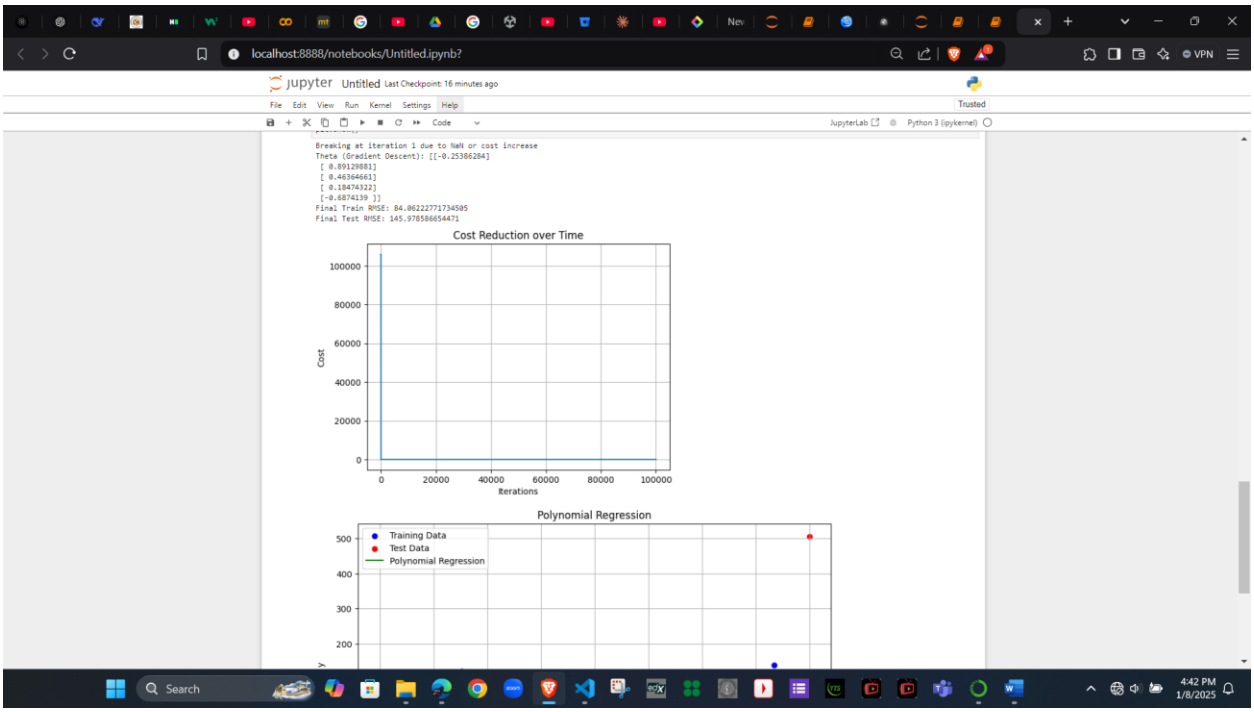


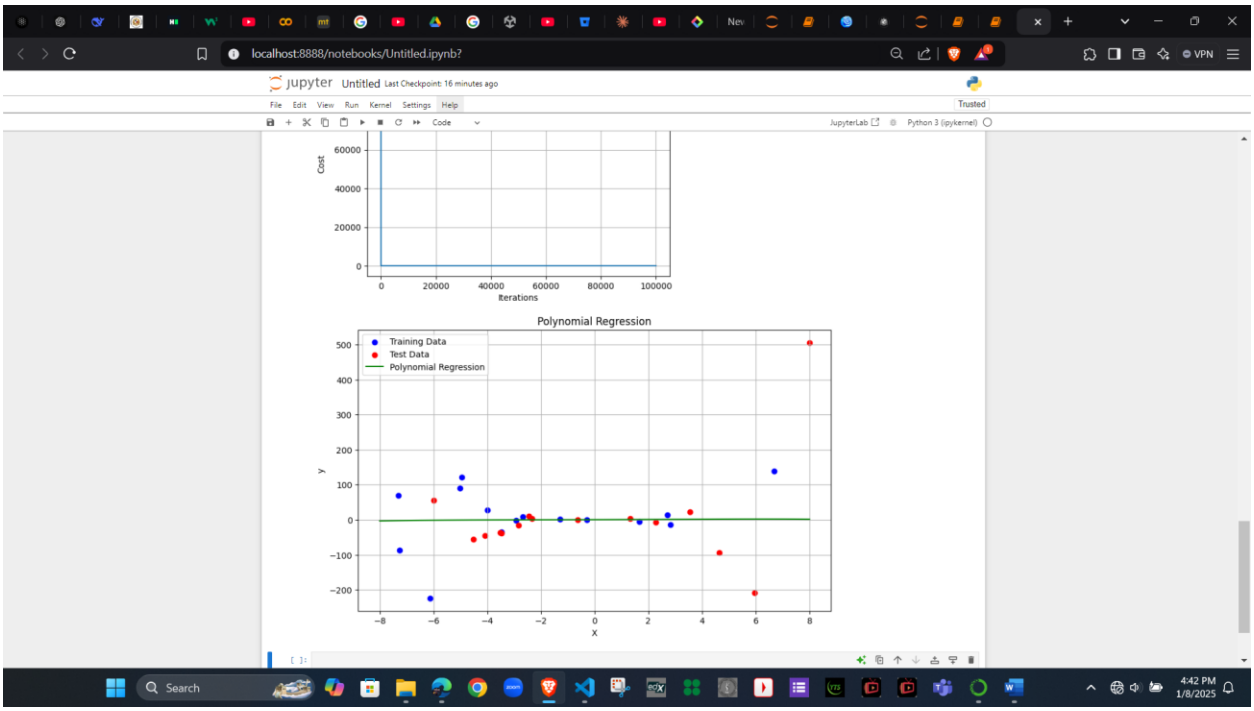


Step 3: Implement gradient descent

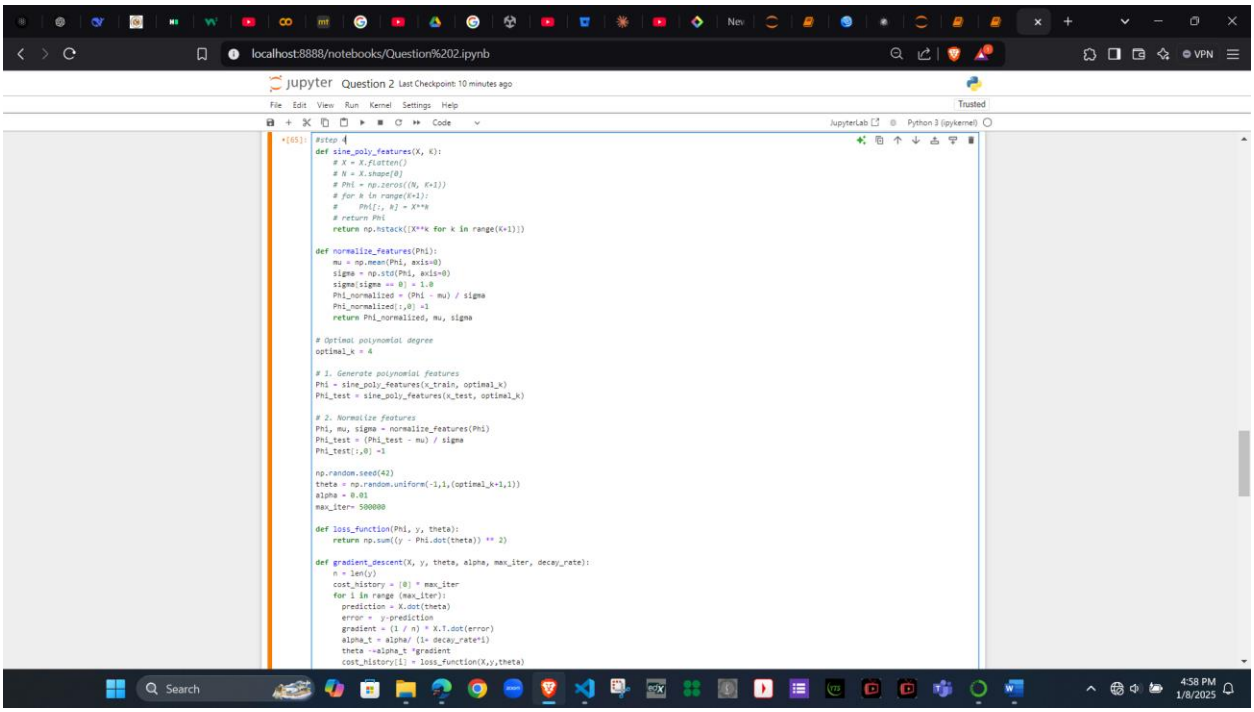


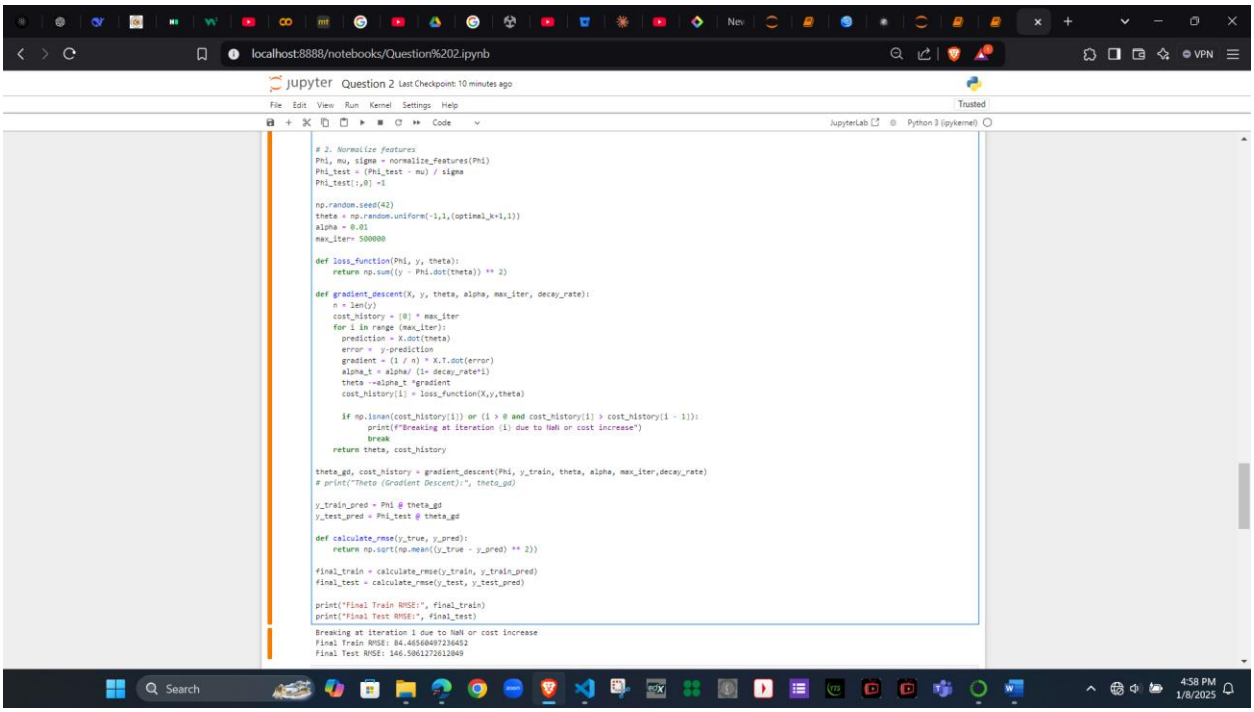

localhost:8888/notebooks/Untitled.ipynb?
jupyter Untitled Last Checkpoint: 16 minutes ago
File Edit View Run Kernel Settings Help
Trusted
JupyterLab Python 3 (ipykernel)
def calculate_rmse(y_true, y_pred):
return np.sqrt(np.mean((y_true - y_pred) ** 2))
final_train_rmse = calculate_rmse(y_train, y_train_pred)
final_test_rmse = calculate_rmse(y_test, y_test_pred)
print("Final Train RMSE:", final_train_rmse)
print("Final Test RMSE:", final_test_rmse)
x_range = np.linspace (-8,8,100).reshape(-1,1)
x_range_poly = sine_poly_features(x_range, optimal_k)
x_range_poly = (x_range_poly - mu) / sigma
x_range_poly[:,0] =1
y_range_pred = x_range_poly @ theta_gd
# Plotting Cost History
plt.figure()
plt.plot(range(len(cost_history)), cost_history)
plt.xlabel("Iterations")
plt.ylabel("Cost")
plt.title("Cost Reduction over Time")
plt.grid(True)
plt.show()
plt.figure(figsize=(10, 6))
plt.scatter(x_train, y_train, label='Training Data', color='blue')
plt.scatter(x_test, y_test, label='Test Data', color='red')
plt.plot(x_range, y_range_pred, label='Polynomial Regression', color='green')
plt.legend()
plt.grid(True)
plt.xlabel('X')
plt.ylabel('y')
plt.title('Polynomial Regression')
plt.show()
Breaking at iteration 1 due to NaN or cost increase
Theta (Gradient Descent): [[-0.25386284]
[0.89129881]
[0.46364661]
[0.18474322]
[-0.6874139]]
Final Train RMSE: 84.06222771734505
Final Test RMSE: 145.97858665447
Cost Reduction over Time
100000
Search
4:42 PM
1/8/2025


localhost:8888/notebooks/Untitled.ipynb?
jupyter Untitled Last Checkpoint: 16 minutes ago
File Edit View Run Kernel Settings Help
Trusted
JupyterLab Python 3 (ipykernel)
Breaking at iteration 1 due to NaN or cost increase
Theta (Gradient Descent): [[-0.25386284]
[0.89129881]
[0.46364661]
[0.18474322]
[-0.6874139]]
Final Train RMSE: 84.06222771734505
Final Test RMSE: 145.97858665447
Cost Reduction over Time
Cost
Iterations
0
20000
40000
60000
80000
100000
Polynomial Regression
Training Data
Test Data
Polynomial Regression
500
400
300
200
Search
4:42 PM
1/8/2025

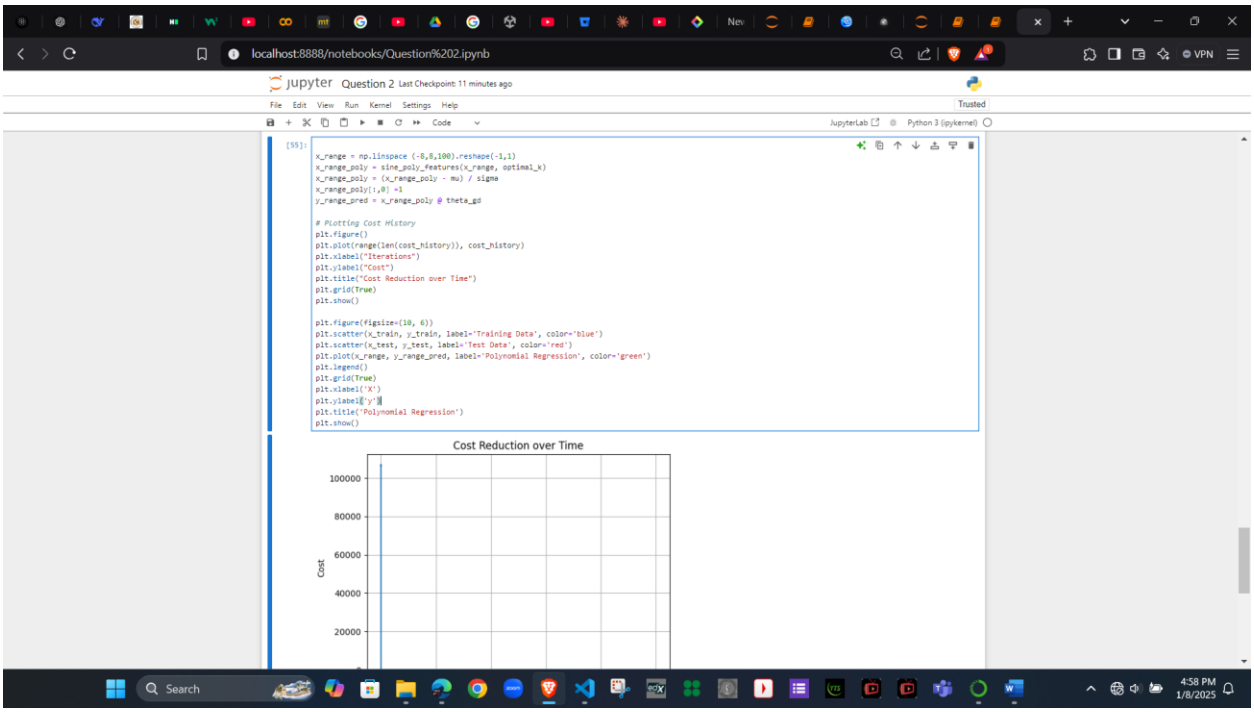

localhost:8888/notebooks/Untitled.ipynb?
jupyter Untitled Last Checkpoint: 16 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
Cost
60000
40000
20000
0
0
20000
40000
60000
80000
100000
Iterations
Polynomial Regression
Training Data
Test Data
Polynomial Regression
y
x
500
400
300
200
100
0
−100
−200
−8
−6
−4
−2
0
2
4
6
8
Search
4:42 PM
1/8/2025

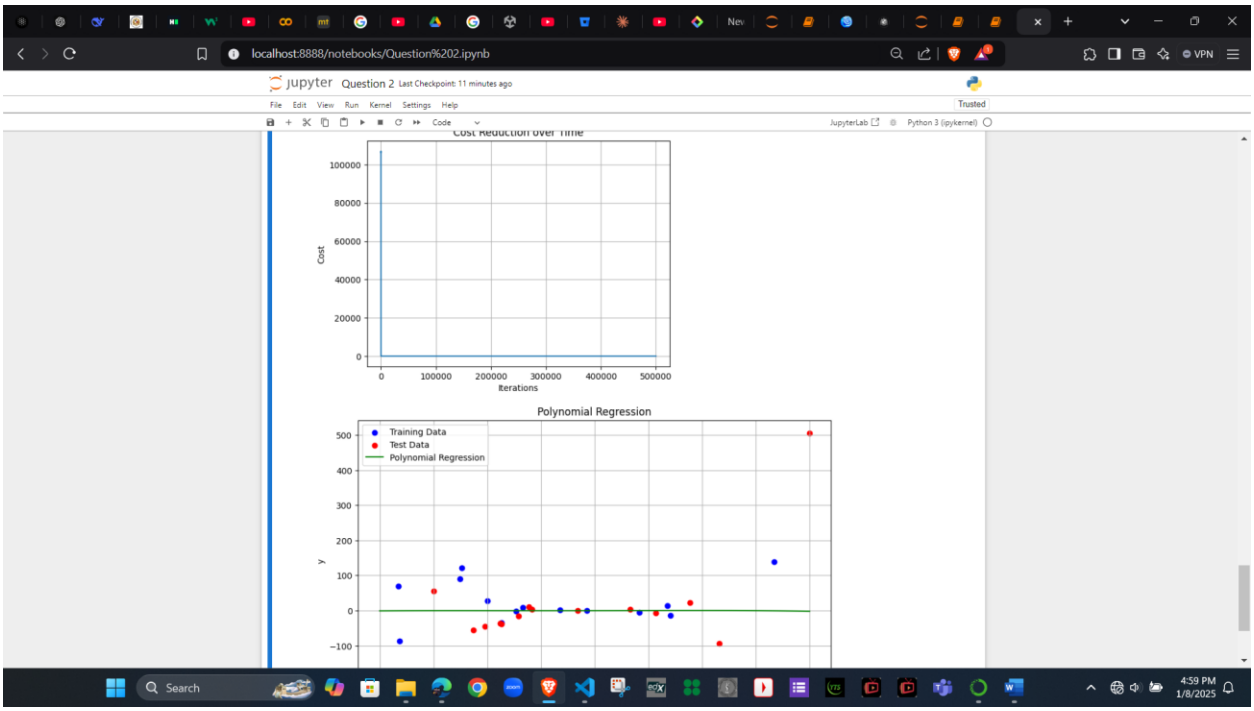
Step 4: Model Evaluation and Comparison

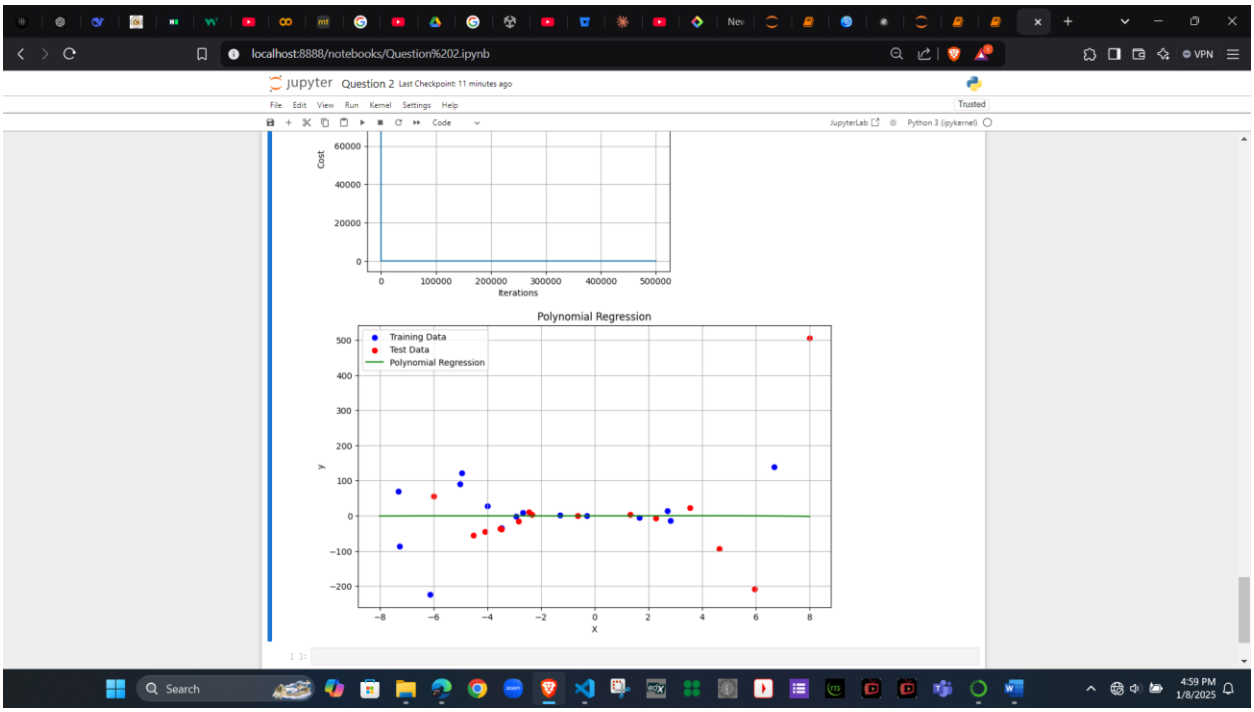




Step 5: Code and analysis



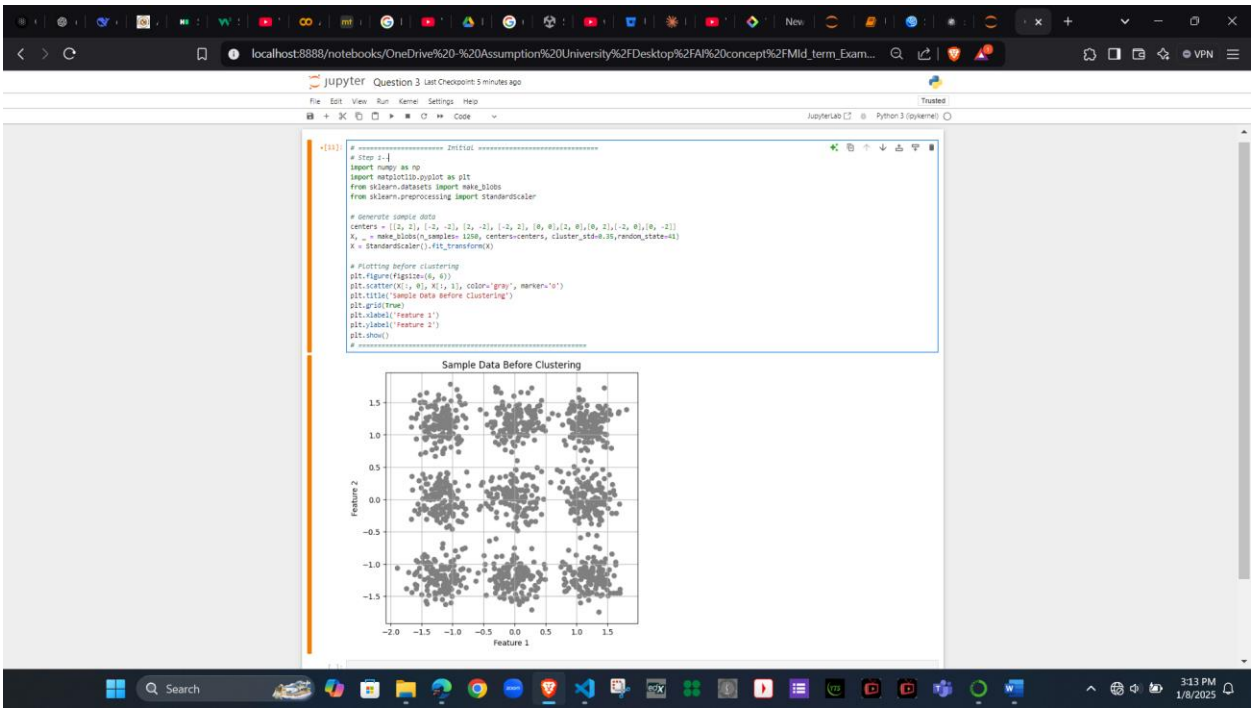

localhost:8888/notebooks/Question%202.ipynb
jupyter Question 2 Last Checkpoint: 11 minutes ago
Cost Reduction over Time
Cost
Iterations
Polynomial Regression
Training Data
Test Data
Polynomial Regression
y
Search
4:59 PM
1/8/2025


localhost:8888/notebooks/Question%202.ipynb
jupyter Question 2 Last Checkpoint: 11 minutes ago
Cost
Iterations
Polynomial Regression
Training Data
Test Data
Polynomial Regression
y
X
Search
4:59 PM
1/8/2025

Question 3

K- means and DB scan clustering

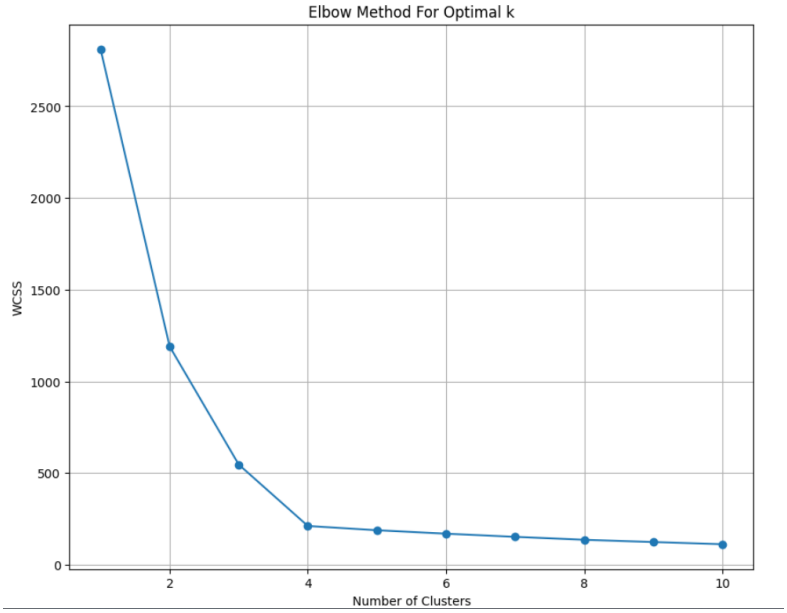
Step 1: Visualize Data Before Clustering



Step 2 : Implement K-means Clustering and Determine Optimal Clusters

```
# Step 2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Generate synthetic data
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

# Calculate WCSS for different numbers of clusters
wcss = []
for i in range(1, 11):  # Testing 1 to 10 clusters
    kmeans = KMeans(n_clusters=i, init='k-means++', max_iter=300, n_init=10,
random_state=0)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)  # inertia_ is the WCSS for the model

print("Kmean values = ",kmeans.inertia_)
# Plotting the WCSS to observe the 'Elbow'
plt.figure(figsize=(10, 8))
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method For Optimal k')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()
```

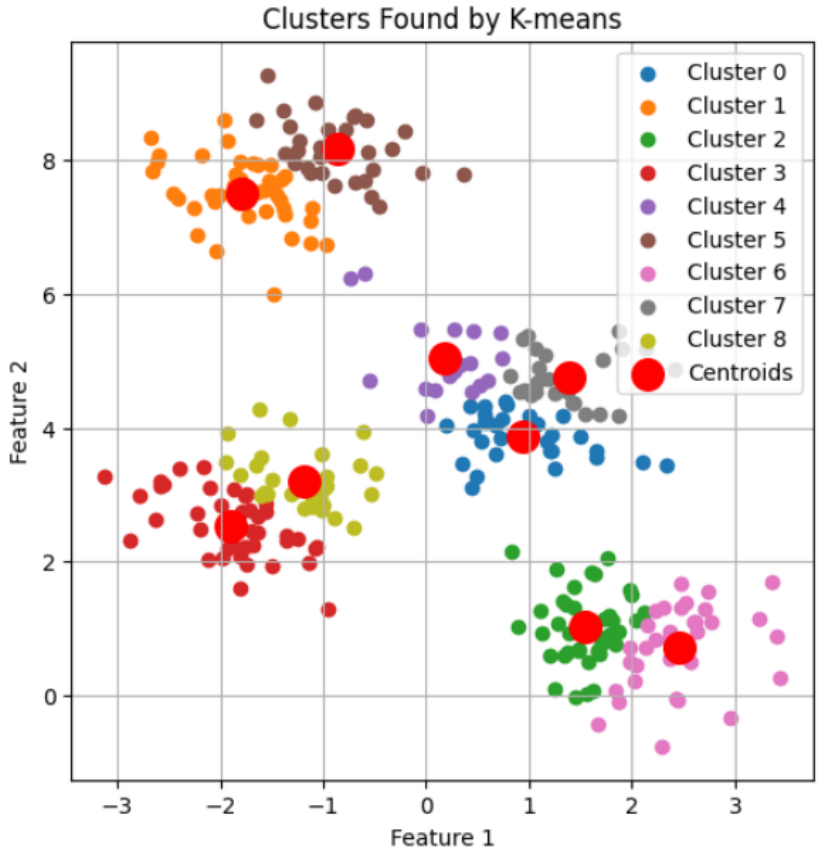
```
Kmean values =  112.58738002972929
```



Kmean values = 112.58738002972929

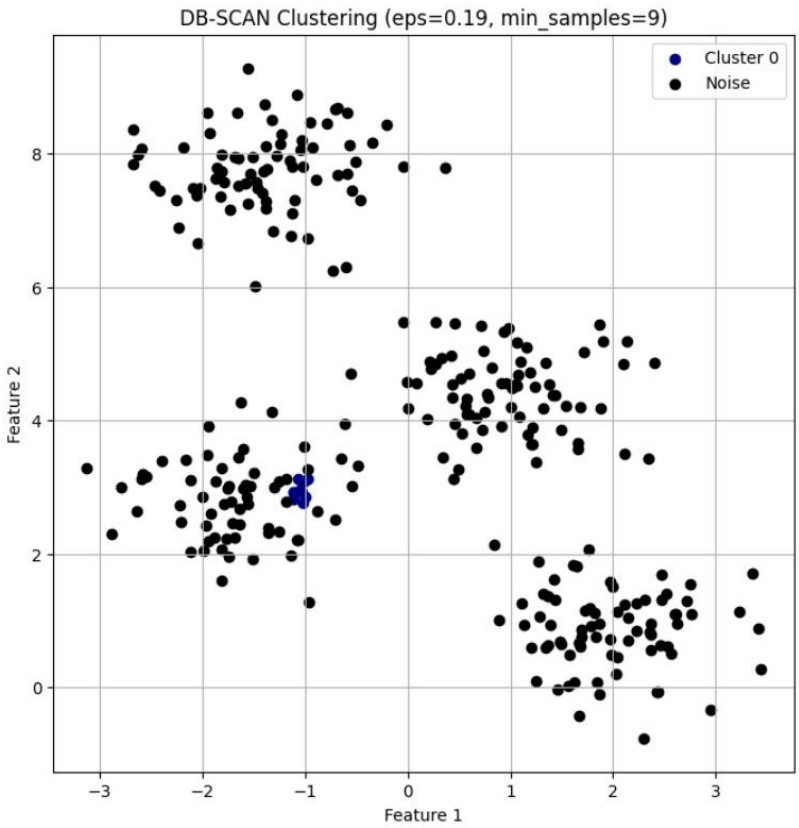
Step 3: Visualizing Data After Clustering

```
# step 3
optimal_k = 9  # Replace with the observed optimal k
kmeans = KMeans(n_clusters=optimal_k, init='k-means++', max_iter=300, n_init=10, random_state=0)
y_kmeans = kmeans.fit_predict(X)

# Plotting clusters
plt.figure(figsize=(6, 6))
for i in range(optimal_k):
    plt.scatter(X[y_kmeans == i, 0], X[y_kmeans == i, 1], label=f'Cluster {i}')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s=200, c='red', label='Centroids')
plt.title('Clusters Found by K-means')
plt.legend()
plt.grid(True)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



Step 4: Applying data from step 1 with DB scan

```python
# Step 4
from sklearn.cluster import DBSCAN

eps = 0.19  # Epsilon parameter
min_samples = 9  # Minimum number of points to form a dense region

db = DBSCAN(eps=eps, min_samples=min_samples).fit(X)
labels = db.labels_

unique_labels = set(labels)

plt.figure(figsize=(8, 8))

# Plot each cluster
for label in unique_labels:
    if label == -1:  # Noise points
        color = 'black'
        label_name = 'Noise'
    else:
        # Use a colormap for clusters
        color = plt.cm.jet(float(label) / len(unique_labels))
        label_name = f'Cluster {label}'

    # Plot points for the current cluster
    plt.scatter(X[labels == label, 0], X[labels == label, 1],
                color=color, label=label_name, marker='o')

plt.title(f'DB-SCAN Clustering (eps={eps}, min_samples={min_samples})')
plt.legend()
plt.grid(True)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



Step 5 : Interpret the results

Interpretation of Results: K-means vs. DB-SCAN

1. K-means Clustering:
    - Output: K-means groups the data into a predefined number of clusters (k). It assigns every data point to the nearest cluster center.
    - Visualization: You can observe well-separated clusters if the data is spherical and evenly distributed.
    - Key Observations:
        - K-means successfully clustered the dataset into distinct groups when the clusters were clearly separated.
        - It assigned all points to clusters, meaning no noise was detected.
        - The elbow method was helpful in determining the optimal number of clusters.
2. DB-SCAN Clustering:
    - Output: DB-SCAN forms clusters based on density and marks points that don't belong to any cluster as noise (-1).
    - Visualization: DB-SCAN effectively separated clusters and identified outliers or noise points in the dataset.
    - Key Observations:
        - DB-SCAN handled non-spherical clusters well and separated clusters based on density rather than distance from the center.
        - It detected noise in areas where the density was too low to form a cluster.

Pros and Cons of Each Algorithm:

1. **K-means Clustering:**
    - Pros:
        - Fast and computationally efficient for large datasets.

- Works well for spherical, well-separated clusters.
- Easy to understand and implement.

- Cons:
  - Requires predefining the number of clusters (k).
  - Sensitive to initialization of centroids and outliers.
  - Poor performance on datasets with non-spherical or overlapping clusters.

2. **DB-SCAN Clustering:**
   - Pros:
     - Can detect clusters of arbitrary shapes.
     - Identifies noise and handles outliers effectively.
     - Does not require the number of clusters to be predefined.
   - Cons:
     - Sensitive to the choice of eps and min_samples parameters.
     - Struggles with datasets that have varying densities.
     - Computationally expensive for large datasets.

Additional code :

```
# Number of clusters found by DB-SCAN (excluding noise)
num_clusters_dbscan = len(set(labels)) - (1 if -1 in labels else 0)
num_noise_points = list(labels).count(-1)

print(f"Number of clusters found by DB-SCAN: {num_clusters_dbscan}")
print(f"Number of noise points identified: {num_noise_points}")

# Compare K-means clusters and DB-SCAN clusters
optimal_k = 5  # Replace with the actual optimal k from the elbow method
kmeans = KMeans(n_clusters=optimal_k, init='k-means++', max_iter=300, n_init=10, random_state=0)
y_kmeans = kmeans.fit_predict(X)

# Evaluate clustering consistency
print(f"Number of clusters found by K-means: {optimal_k}")
```

```
Number of clusters found by DB-SCAN: 1
Number of noise points identified: 290
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1382: UserWarning: KMeans is known
there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NU
  warnings.warn(
Number of clusters found by K-means: 5
```

Number of clusters found by DB-SCAN: 1

Number of noise points identified: 290

Number of clusters found by K-means: 5